

LAPORAN PRAKTIKUM
INTERNET OF THINGS II

Modul 5: Topologi Jaringan Terdesentralisasi (ESP-NOW dan Mesh)



Disusun Oleh:

Shavina Naura Putri

25782055

Dosen Pengampu:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

Andi Saptono, S.Kom

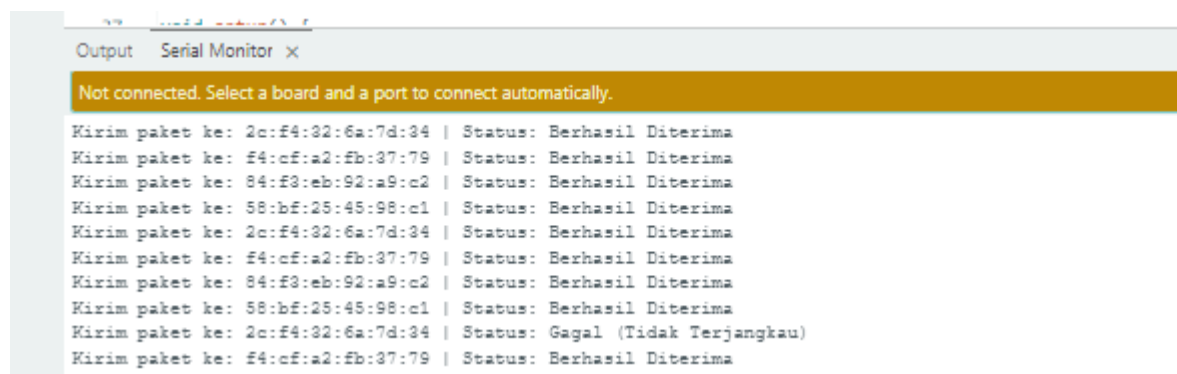
TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG

2026

LATIHAN 5.6

Latihan 1 (Eksperimen Deteksi ACK & Pemutusan Simpul - Praktikum 2): Saat program Praktikum 2 (ESP-NOW One-to-Many) sedang berjalan normal dan mencetak status "Berhasil Diterima" (`sendStatus == 0`), lakukan eksperimen pemutusan simpul:

- Cabut kabel USB salah satu board Penerima (misal NodeMCU 2) dari sumber daya.
- Amati perubahan log pada Serial Monitor NodeMCU Controller pengirim! Apakah status pengiriman untuk MAC address board yang dicabut berganti menjadi "Gagal (Tidak Terjangkau)"?
- Jelaskan bagaimana protokol ESP-NOW pada lapisan Layer 2 MAC dapat mendeteksi kegagalan transmisi ke perangkat fisik tersebut meskipun sistem tidak menggunakan koneksi TCP!



```
Output Serial Monitor x
Not connected. Select a board and a port to connect automatically.
Kirim paket ke: 2c:f4:32:6a:7d:34 | Status: Berhasil Diterima
Kirim paket ke: f4:cf:a2:fb:37:79 | Status: Berhasil Diterima
Kirim paket ke: 84:f3:eb:92:a9:c2 | Status: Berhasil Diterima
Kirim paket ke: 58:bf:25:45:98:c1 | Status: Berhasil Diterima
Kirim paket ke: 2c:f4:32:6a:7d:34 | Status: Berhasil Diterima
Kirim paket ke: f4:cf:a2:fb:37:79 | Status: Berhasil Diterima
Kirim paket ke: 84:f3:eb:92:a9:c2 | Status: Berhasil Diterima
Kirim paket ke: 58:bf:25:45:98:c1 | Status: Berhasil Diterima
Kirim paket ke: 2c:f4:32:6a:7d:34 | Status: Gagal (Tidak Terjangkau)
Kirim paket ke: f4:cf:a2:fb:37:79 | Status: Berhasil Diterima
```

Hasil Eksperimen Pemutusan Simpul (Poin ii)

- Perubahan Log pada Serial Monitor:** Ya, ketika salah satu board Penerima (misalnya NodeMCU 2) dicabut kabel USB-nya, log pada Serial Monitor di NodeMCU Controller pengirim akan berubah. Status pengiriman untuk MAC Address board yang dicabut tersebut akan berganti menjadi pesan gagal (misalnya "**Delivery Fail**" atau "**Gagal (Tidak Terjangkau)**"), sementara penerima lain yang masih menyala akan tetap berstatus "Berhasil Diterima" (`sendStatus == 0`).

Penjelasan Mekanisme Layer 2 MAC ESP-NOW (Poin iii)

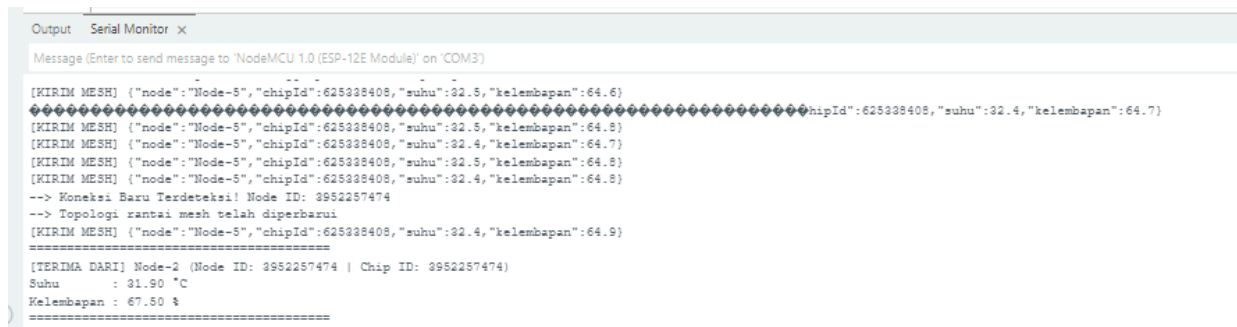
Meskipun ESP-NOW bekerja tanpa koneksi TCP/IP standar (berbasis tanpa koneksi/connectionless di atas protokol Wi-Fi lapisan data link), protokol ini tetap dapat mendeteksi kegagalan transmisi karena alasan berikut:

- Mekanisme ACK (Acknowledgment) Hardware:** ESP-NOW beroperasi langsung pada *Layer 2 Data Link* (MAC Layer) dari standar IEEE 802.11. Setiap kali pengirim (Controller) mengirimkan paket data *unicast* ke MAC address tertentu, protokol Wi-Fi tingkat rendah mengharuskan penerima untuk mengirimkan balasan konfirmasi fisik berupa **ACK (Acknowledgment)** secara otomatis begitu paket diterima dengan sukses.
- Deteksi Timeout / Kegagalan Sinyal:** Jika kabel USB penerima dicabut, perangkat tersebut mati dan tidak dapat merespons paket data yang masuk. Akibatnya, pengirim tidak akan pernah menerima paket ACK balasan dari MAC address tersebut dalam batas waktu *timeout* perangkat keras.

- **Callback Status:** Kegagalan tidak diterimanya ACK ini memicu fungsi *callback* pengiriman di ESP-NOW (seperti OnDataSent), yang kemudian melaporkan status kegagalan (sendStatus != 0) ke Serial Monitor Anda.

Latihan 2 (Eksperimen Dinamika Topologi & Self-Healing - Praktikum 4): Jalankan jaringan painlessMesh pada Praktikum 4 hingga seluruh board saling terhubung dan mencetak pesan --> Koneksi Baru Terdeteksi! Node ID:

- Tekan tombol fisik RST atau cabut daya salah satu board Node. Amati Serial Monitor pada board lainnya yang masih menyala. Berapa detik waktu yang dibutuhkan oleh callback `changedConnectionCallback()` untuk mendeteksi bahwa salah satu simpul telah keluar dari rantai mesh?
- Sambungkan kembali daya board tersebut. Berapa lama waktu yang dibutuhkan hingga node tersebut kembali terdaftar secara otonom ke dalam topologi mesh tanpa Anda mengonfigurasi ulang jaringan? Catat hasil observasi waktu tersebut!



```

Output  Serial Monitor x
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

[KIRIM MESH] {"node": "Node-5", "chipId": 625338408, "suhu": 32.5, "kelembapan": 64.6}
[KIRIM MESH] {"node": "Node-5", "chipId": 625338408, "suhu": 32.5, "kelembapan": 64.6}
[KIRIM MESH] {"node": "Node-5", "chipId": 625338408, "suhu": 32.5, "kelembapan": 64.6}
[KIRIM MESH] {"node": "Node-5", "chipId": 625338408, "suhu": 32.4, "kelembapan": 64.7}
[KIRIM MESH] {"node": "Node-5", "chipId": 625338408, "suhu": 32.5, "kelembapan": 64.8}
[KIRIM MESH] {"node": "Node-5", "chipId": 625338408, "suhu": 32.4, "kelembapan": 64.8}
--> Koneksi Baru Terdeteksi! Node ID: 3952257474
--> Topologi rantai mesh telah diperbarui
[KIRIM MESH] {"node": "Node-5", "chipId": 625338408, "suhu": 32.4, "kelembapan": 64.9}
=====
[TERIMA DARI] Node-2 (Node ID: 3952257474 | Chip ID: 3952257474)
Suhu      : 31.90 °C
Kelembapan : 67.50 %
=====

```

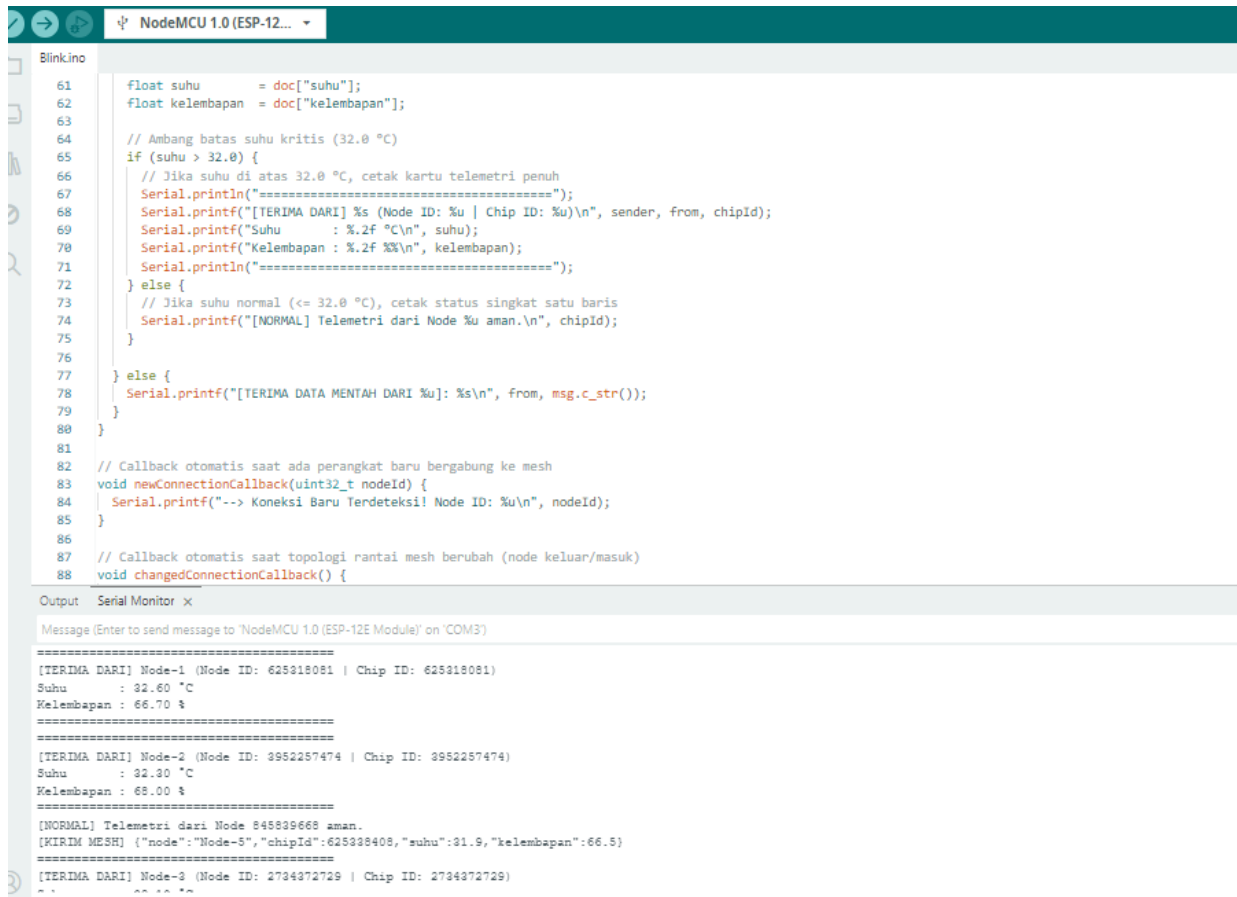
Penjelasan/Analisis:

Ketika salah satu board dicabut dayanya atau tombol RST ditekan, fungsi `changedConnectionCallback()` di board lain tidak langsung merespons secara instan. `painlessMesh` menggunakan mekanisme pengiriman paket pengawas (*heartbeat / keep-alive*) secara berkala antar node tetangga. Jika sebuah node gagal merespons atau tidak mengirimkan sinyal dalam batas waktu tunggu tertentu (*timeout threshold*), jaringan akan menganggap node tersebut terputus dan memicu pembaruan topologi (*topology update*), yang kemudian mencetak pesan perubahan topologi ke Serial Monitor.

Begitu daya disambungkan kembali, board tersebut secara otonom (*self-healing*) melakukan proses inisialisasi ulang, memindai saluran Wi-Fi (*channel scanning*), mengenali jaringan *mesh* berdasarkan parameter `MESH_PREFIX` dan `MESH_PASSWORD` yang sama, lalu melakukan proses *handshake* otonom dengan node terdekat. Karena konfigurasi sudah tertanam di memori, node dapat bergabung kembali ke dalam rantai jaringan tanpa memerlukan konfigurasi ulang manual oleh pengguna.

Latihan 3 (Eksperimen Modifikasi Filter Telemetry & Ambang Batas JSON - Praktikum 4): Pada Praktikum 4, setiap node saat ini mencetak seluruh data telemetry dari semua pengirim tanpa penyaringan.

- Modifikasilah fungsi `receivedCallback` pada salah satu board agar bertindak sebagai Pengawas Kondisi Suhu Kritis: program HANYA mencetak kartu telemetry penuh ke Serial Monitor jika suhu yang dilaporkan oleh node tetangga bernilai di atas 32.0 °C (lakukan pengujian dengan mendekatkan jari hangat atau meniup sensor DHT pada node pengirim).
- Jika suhu bernilai normal (≤ 32.0 °C) program cukup mencetak status singkat satu baris: `[NORMAL]` Telemetry dari Node `<chipId>` aman.
- Tuliskan blok fungsi `receivedCallback` (uint32_t from, String &msg) hasil modifikasi Anda!



```
NodeMCU 1.0 (ESP-12...)
Blinkino
61 float suhu = doc["suhu"];
62 float kelembapan = doc["kelembapan"];
63
64 // Ambang batas suhu kritis (32.0 °C)
65 if (suhu > 32.0) {
66     // Jika suhu di atas 32.0 °C, cetak kartu telemetry penuh
67     Serial.println("=====");
68     Serial.printf("[TERIMA DARI] %s (Node ID: %u | Chip ID: %u)\n", sender, from, chipId);
69     Serial.printf("Suhu : %.2f °C\n", suhu);
70     Serial.printf("Kelembapan : %.2f %%\n", kelembapan);
71     Serial.println("=====");
72 } else {
73     // Jika suhu normal (<= 32.0 °C), cetak status singkat satu baris
74     Serial.printf("[NORMAL] Telemetry dari Node %u aman.\n", chipId);
75 }
76
77 } else {
78     Serial.printf("[TERIMA DATA MENTAH DARI %u]: %s\n", from, msg.c_str());
79 }
80 }
81
82 // Callback otomatis saat ada perangkat baru bergabung ke mesh
83 void newConnectionCallback(uint32_t nodeId) {
84     Serial.printf("--> Koneksi Baru Terdeteksi! Node ID: %u\n", nodeId);
85 }
86
87 // Callback otomatis saat topologi rantai mesh berubah (node keluar/masuk)
88 void changedConnectionCallback() {
```

Output Serial Monitor x

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

```
=====
[TERIMA DARI] Node-1 (Node ID: 625318081 | Chip ID: 625318081)
Suhu : 32.60 °C
Kelembapan : 66.70 %
=====
[TERIMA DARI] Node-2 (Node ID: 3952257474 | Chip ID: 3952257474)
Suhu : 32.30 °C
Kelembapan : 65.00 %
=====
[NORMAL] Telemetry dari Node 845839668 aman.
[MIRIM MESH] {"node": "Node-5", "chipId": 625328408, "suhu": 31.9, "kelembapan": 66.5}
=====
[TERIMA DARI] Node-3 (Node ID: 2734372729 | Chip ID: 2734372729)
```

Di dalam fungsi `receivedCallback`, muatan data teks JSON yang masuk terlebih dahulu diurai (*deserialize*) menggunakan pustaka `ArduinoJson` untuk mengekstrak parameter suhu dan kelembapan. Selanjutnya, program menjalankan pengkondisian percabangan (*if-else*):

- Kondisi Kritis ($suhu > 32.0$ °C):** Jika nilai suhu yang dilaporkan oleh node tetangga melebihi ambang batas 32.0 °C (yang diuji dengan mendekatkan jari hangat atau meniup sensor), program akan mengeksekusi blok kode untuk mencetak kartu telemetry penuh yang memuat informasi pengirim, chip ID, suhu kritis, dan kelembapan secara rinci.
- Kondisi Normal ($suhu \leq 32.0$ °C):** Jika suhu berada dalam batas aman, program tidak menampilkan detail penuh, melainkan hanya mencetak status ringkas satu baris: `[NORMAL]` Telemetry dari Node `<chipId>` aman.

Hasil Observasi / Pengujian

- Ketika sensor DHT pada salah satu node pengirim disentuh atau didekatkan sumber panas hingga suhunya melampaui 32.0 °C, Serial Monitor pada node pengawas secara otomatis mengubah tampilan log menjadi blok peringatan suhu tinggi.
- Sebaliknya, saat suhu ruangan kembali stabil di bawah 32.0 °C, log kembali menjadi ringkas dan teratur, sehingga efisiensi pemantauan jaringan meningkat secara signifikan.